

**LAPORAN UJIAN TENGAH SEMESTER
MATA KULIAH SOFTWARE ENGINEER
“SISTEM PENGADUAN MAHASISWA ”**



Disusun oleh:

Muhammad Fachri Wiryansyah

2410511096

**PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN” JAKARTA
2025**

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat telah mendorong berbagai institusi, termasuk perguruan tinggi, untuk mengadopsi sistem digital dalam mendukung aktivitas operasionalnya. Salah satu kebutuhan penting dalam lingkungan akademik adalah adanya sistem pengaduan (complaint system) yang dapat digunakan oleh mahasiswa untuk menyampaikan keluhan, aspirasi, maupun laporan terkait layanan kampus secara cepat, transparan, dan terdokumentasi dengan baik.

Namun, pada banyak implementasi sistem tradisional yang masih menggunakan pendekatan monolitik, terdapat berbagai kendala seperti sulitnya pengembangan fitur baru, keterbatasan dalam skalabilitas sistem, serta tingginya risiko kegagalan sistem secara keseluruhan ketika terjadi error pada satu bagian. Hal ini menyebabkan sistem menjadi kurang fleksibel dan tidak mampu beradaptasi dengan kebutuhan pengguna yang terus berkembang.

Untuk mengatasi permasalahan tersebut, pendekatan arsitektur microservices menjadi salah satu solusi yang relevan. Dengan memecah sistem menjadi beberapa layanan kecil yang saling terhubung, setiap layanan dapat dikembangkan, dikelola, dan di-deploy secara independen. Selain itu, penggunaan API Gateway memungkinkan pengelolaan request secara terpusat, sehingga meningkatkan keamanan dan efisiensi komunikasi antar layanan.

Dalam implementasinya, sistem ini juga memanfaatkan teknologi autentikasi modern seperti JSON Web Token (JWT) dan OAuth berbasis Google untuk menjamin keamanan akses pengguna. Selain itu, komunikasi antar layanan (inter-service communication) memungkinkan integrasi antar modul seperti pengaduan, disposisi, dan notifikasi, sehingga sistem dapat berjalan secara terintegrasi namun tetap modular.

Berdasarkan latar belakang tersebut, maka pada tugas ini dibangun sebuah sistem pengaduan berbasis microservices yang terdiri dari beberapa layanan utama, yaitu Auth Service, Complaint Service, Disposition Service, dan Notification Service, yang diintegrasikan melalui API Gateway.

1.2 Tujuan

Adapun tujuan dari pembuatan sistem ini adalah sebagai berikut:

1. Membangun sistem pengaduan berbasis arsitektur microservices yang modular dan scalable.
2. Mengimplementasikan API Gateway sebagai pintu masuk utama untuk mengelola routing dan keamanan request.
3. Menerapkan sistem autentikasi menggunakan JSON Web Token (JWT) untuk menjaga keamanan akses pengguna.
4. Mengintegrasikan fitur login menggunakan OAuth Google sebagai alternatif autentikasi modern.
5. Mengembangkan layanan complaint yang mampu menangani pembuatan, pengelolaan, dan pencarian data pengaduan.
6. Mengimplementasikan layanan disposition untuk mengelola proses penanganan dan penerusan pengaduan.
7. Membangun layanan notification untuk memberikan informasi kepada pengguna secara otomatis terkait perubahan status pengaduan.
8. Mengimplementasikan komunikasi antar layanan (inter-service communication) menggunakan REST API.
9. Membandingkan kelebihan arsitektur microservices dibandingkan dengan arsitektur monolitik dalam pengembangan sistem.

ARSITEKTUR SISTEM

2.1 Gambaran Umum Sistem

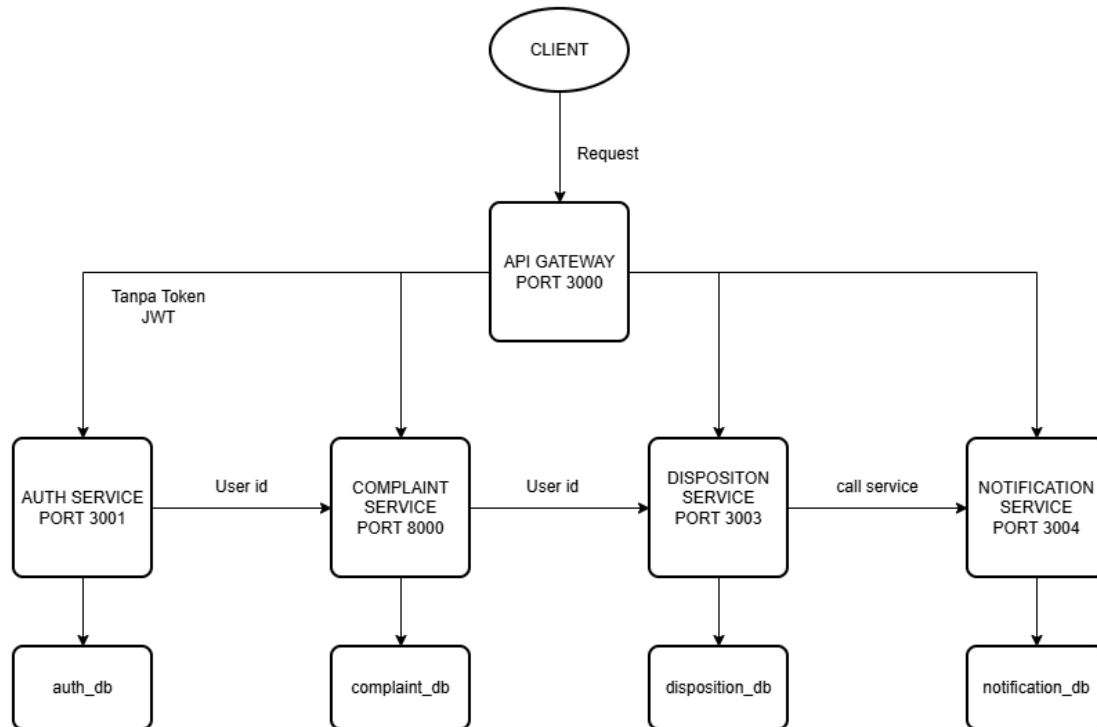
Sistem yang dibangun merupakan aplikasi pengaduan mahasiswa berbasis arsitektur microservices, di mana setiap fungsi utama sistem dipisahkan ke dalam layanan (service) yang berdiri sendiri dan saling berkomunikasi melalui API.

Sistem ini terdiri dari lima komponen utama, yaitu:

1. API Gateway sebagai pintu masuk utama seluruh request dari client
2. Auth Service untuk autentikasi dan manajemen pengguna
3. Complaint Service untuk pengelolaan data pengaduan
4. Disposition Service untuk proses tindak lanjut pengaduan
5. Notification Service untuk pengiriman notifikasi kepada pengguna

Setiap service memiliki database masing-masing sehingga tidak terjadi ketergantungan langsung antar database, dan komunikasi antar service dilakukan melalui REST API.

2.2 Diagram Arsitektur Sistem



Penjelasan alur:

1. Client mengirim request ke API Gateway
2. API Gateway memverifikasi token JWT (kecuali endpoint auth)
3. Request diteruskan ke service tujuan
4. Service dapat berkomunikasi dengan service lain jika diperlukan
5. Response dikembalikan ke client melalui Gateway

2.3 Penjelasan Tiap Service

2.3.1 Auth Service

Auth Service bertanggung jawab dalam proses autentikasi dan manajemen pengguna. Fitur utama yang disediakan meliputi registrasi pengguna, login menggunakan email dan password, login melalui OAuth Google, pembuatan access token dan refresh token, logout serta penghapusan akun, dan pengambilan profil pengguna.

Implementasi teknis:

- Menggunakan Node.js dan Express
- Database: MySQL (auth_db)
- Password di-hash menggunakan bcrypt
- Token dibuat menggunakan JWT

Mekanisme autentikasi:

1. User melakukan login dan sistem memverifikasi kredensial.
2. Sistem menghasilkan access token dan refresh token.
3. Token dikirim ke client.
4. Token digunakan untuk mengakses service lain.

Pada fitur OAuth Google, sistem menggunakan *passport-google-oauth20*. Pengguna dapat login melalui akun Google, dan data pengguna akan disimpan apabila belum terdaftar. Setelah proses autentikasi berhasil, sistem menghasilkan JWT.

2.3.2 Complaint Service

Complaint Service bertugas mengelola data pengaduan, termasuk pembuatan, pengambilan, pembaruan, dan penghapusan data. Service ini juga mendukung fitur pagination dan filtering serta memiliki relasi dengan kategori, status, dan rating.

Fitur:

- Create complaint
- View complaint (pagination & filter)

- Update complaint
- Delete complaint
- Relasi dengan category, status, dan rating

Implementasi:

- Menggunakan Laravel (PHP MVC)
- Database: complaint_db
- Menggunakan Eloquent ORM

Integrasi dengan Auth Service:

- Tidak memiliki tabel user sendiri
- Mengambil user dari Auth Service melalui endpoint /auth/profile
- User ID disimpan sebagai referensi

2.3.3 Disposition Service

Disposition Service bertugas mengelola proses tindak lanjut pengaduan. Service ini memungkinkan penerusan complaint ke unit tertentu, pembaruan status penanganan, serta pencatatan log aktivitas.

Fitur:

- Meneruskan complaint ke unit tertentu
- Update status penanganan
- Menyimpan log aktivitas

Implementasi:

- Node.js + Express
- Database: disposition_db

Integrasi:

1. Mengecek validitas complaint ke Complaint Service
2. Mengambil user_id dari complaint
3. Mengirim notifikasi ke Notification Service

2.3.4 Notification Service

Notification Service bertanggung jawab untuk menyimpan dan menyediakan notifikasi kepada pengguna. Notifikasi dihasilkan secara otomatis saat terjadi perubahan status atau disposisi pengaduan.

Fitur:

- Menyimpan notifikasi
- Mengambil notifikasi berdasarkan user

Implementasi:

- Node.js + Express
- Database: notification_db

Integrasi:

- Dipanggil oleh Disposition Service
- Notifikasi dibuat otomatis saat:
 - Complaint didisposisikan
 - Status berubah

2.3.5 API Gateway

API Gateway berfungsi sebagai penghubung antara client dan seluruh service. Gateway bertindak sebagai *single entry point* yang mengatur routing, validasi JWT, serta pembatasan request.

Fungsi utama:

- Routing request ke service yang sesuai
- Verifikasi JWT
- Rate limiting
- Single entry point

Implementasi:

- Node.js + Express
- Menggunakan http-proxy-middleware

Routing:

- /auth → auth-service
- /complaints → complaint-service
- /dispositions → disposition-service
- /notifications → notification-service

2.4 Alur Sistem

1. Login

- User login ke auth-service dapat menggunakan OAuth dengan akun google
- Mendapatkan access token

2. Membuat Complaint

- Request dikirim ke gateway
- Gateway validasi token
- Complaint disimpan dengan user_id dari auth-service

3. Disposition

- Disposition service:
 - cek complaint ke complaint-service
 - simpan disposition
 - kirim notifikasi

4. Notification

- Notification service menyimpan notifikasi
- User dapat melihat notifikasi berdasarkan user_id

2.4 Justifikasi Arsitektur Microservice

Pada sistem yang dikembangkan, arsitektur microservice dipilih dibandingkan pendekatan monolitik untuk meningkatkan modularitas, skalabilitas, dan kemudahan pengelolaan sistem. Setiap layanan dirancang untuk menangani satu tanggung jawab utama sehingga perubahan pada satu fitur tidak berdampak langsung pada fitur lainnya.

Auth Service dipisahkan karena berfungsi sebagai pusat autentikasi dan otorisasi pengguna. Dengan memisahkan layanan ini, sistem dapat memastikan keamanan yang lebih baik serta memungkinkan penggunaan ulang (reusability) oleh service lain tanpa harus menduplikasi logika autentikasi.

Complaint Service berperan sebagai inti dari sistem, yaitu menangani data pengaduan pengguna. Service ini menggunakan pendekatan MVC dengan framework Laravel sehingga struktur kode lebih terorganisir. Pemisahan ini memungkinkan pengelolaan data pengaduan secara independen dari fitur lain seperti autentikasi atau notifikasi.

Disposition Service dibuat untuk menangani proses lanjutan dari pengaduan, yaitu pendistribusian ke unit terkait. Pemisahan ini penting karena logika bisnisnya berbeda dari complaint dan dapat berkembang menjadi lebih kompleks, misalnya penambahan workflow atau approval system.

Notification Service dipisahkan untuk menangani pengiriman notifikasi kepada pengguna. Dengan memisahkan service ini, sistem menjadi lebih fleksibel karena notifikasi dapat dikembangkan secara asynchronous di masa depan tanpa mengganggu service utama.

API Gateway digunakan sebagai pintu masuk tunggal (single entry point) untuk seluruh request dari client. Gateway bertugas mengatur routing, validasi JWT, dan pembatasan request (rate limiting), sehingga setiap service tidak perlu menangani hal tersebut secara berulang.

Dengan pendekatan ini, sistem menjadi lebih scalable, maintainable, dan sesuai dengan prinsip loose coupling dalam arsitektur modern.

2.5 API Gateway Routing Map

API Gateway berfungsi sebagai penghubung antara client dengan seluruh microservice yang ada. Semua request dari client harus melalui gateway, sehingga client tidak berinteraksi langsung dengan service di belakangnya.

Routing yang digunakan dalam sistem ini adalah sebagai berikut:

Endpoint Gateway	Service Tujuan	Keterangan
/auth/*	Auth Service (port 3001)	Login, register, OAuth
/complaints/*	Complaint Service (port 8000)	Kelola pengaduan
/dispositions/*	Disposition Service (port 3003)	Proses distribusi
/notifications/*	Notification Service (port 3004)	Ambil notifikasi

Gateway juga melakukan:

- Validasi JWT sebelum request diteruskan (kecuali auth)
- Rate limiting (60 request per menit)
- Routing berdasarkan path

Dengan adanya gateway, sistem menjadi lebih aman dan terpusat dalam pengelolaan akses.

2.6 Alur Sistem

Alur kerja sistem menggambarkan bagaimana setiap service saling berinteraksi dalam menangani proses pengaduan.

Pertama, pengguna melakukan login melalui Auth Service. Setelah berhasil, sistem akan memberikan access token (JWT) yang digunakan untuk mengakses endpoint lain.

Selanjutnya, pengguna membuat pengaduan melalui Complaint Service. Pada proses ini, Complaint Service akan memvalidasi token dengan Auth Service untuk memastikan identitas pengguna, kemudian menyimpan data pengaduan beserta user_id yang diperoleh dari Auth Service.

Setelah pengaduan dibuat, admin atau sistem dapat membuat disposition melalui Disposition Service. Service ini akan terlebih dahulu memvalidasi keberadaan complaint dengan memanggil Complaint Service.

Jika disposition berhasil dibuat, maka Disposition Service akan memanggil Notification Service untuk mengirimkan notifikasi kepada pengguna terkait bahwa pengaduannya telah diproses atau diteruskan.

Ketika status disposition diperbarui, sistem kembali mengirim notifikasi kepada pengguna sebagai bentuk update informasi.

Akhirnya, pengguna dapat melihat notifikasi melalui Notification Service berdasarkan user_id yang dimilikinya.

Alur ini menunjukkan adanya komunikasi antar service (inter-service communication) yang menjadi salah satu ciri utama arsitektur microservice.

DATABASE DESIGN

3.1 Konsep Desain Database

Pada sistem ini, setiap microservice memiliki basis data yang terpisah sesuai dengan prinsip database per service. Pendekatan ini bertujuan untuk menghindari ketergantungan langsung antar layanan (tight coupling) serta meningkatkan fleksibilitas dalam pengembangan dan pengelolaan sistem.

Setiap service hanya dapat mengakses databasenya sendiri, sedangkan kebutuhan data dari service lain diperoleh melalui mekanisme komunikasi antar service (API). Dengan demikian, sistem menjadi lebih modular dan mudah untuk dikembangkan secara independen.

3.2 Struktur Database per Service

Auth Service (auth_db)

Database ini digunakan untuk mengelola data pengguna dan autentikasi.

Tabel utama:

- users → menyimpan data pengguna (nama, email, password, OAuth)
- refresh_tokens → menyimpan token untuk sesi login

Relasi sederhana:

users (1) ——— (N) refresh_tokens

Penjelasan:

Satu user dapat memiliki beberapa refresh token karena user bisa login dari beberapa perangkat.

Complaint Service (complaint_db)

Database ini menangani data pengaduan mahasiswa.

Tabel utama:

- complaints
- categories
- statuses
- ratings

Relasi:

categories (1) — (N) complaints

statuses (1) — (N) complaints

complaints (1) — (1) ratings

Penjelasan:

- Setiap complaint memiliki satu kategori dan satu status
- Setiap complaint dapat memiliki satu rating
- Data user tidak disimpan di sini, hanya menggunakan user_id sebagai referensi dari auth-service

Disposition Service (disposition_db)

Database ini digunakan untuk proses tindak lanjut pengaduan.

Tabel utama:

- dispositions
- units
- logs

Relasi:

units (1) — (N) dispositions

dispositions (1) — (N) logs

Penjelasan:

- Setiap disposition dikirim ke satu unit
- Setiap perubahan status dicatat dalam tabel log
- complaint_id hanya sebagai referensi dari complaint-service

Notification Service (notification_db)

Database ini menyimpan notifikasi pengguna.

Tabel utama:

- notifications

Struktur sederhana:

notifications (user_id, title, message, is_read)

Penjelasan:

- user_id berasal dari auth-service
- Tidak ada relasi langsung ke database lain

3.3 Desain Relasi Database

Setiap microservice dalam sistem ini menggunakan database yang terpisah untuk menjaga independensi data dan menghindari ketergantungan langsung antar service.

Pada Complaint Service, digunakan database relasional yang terdiri dari beberapa tabel utama yaitu:

- Tabel complaints sebagai tabel utama yang menyimpan data pengaduan pengguna
- Tabel categories untuk mengelompokkan jenis pengaduan
- Tabel statuses untuk menyimpan status pengaduan (misalnya: pending, diproses, selesai)
- Tabel ratings untuk menyimpan penilaian terhadap penyelesaian pengaduan

Relasi antar tabel tersebut adalah sebagai berikut:

- Setiap complaint memiliki satu category (many-to-one)
- Setiap complaint memiliki satu status (many-to-one)
- Setiap complaint dapat memiliki satu rating (one-to-one)

Selain itu:

- Disposition Service memiliki tabel:
 - dispositions (menghubungkan complaint_id dan unit_id)
 - units (unit tujuan)
 - logs (riwayat aktivitas)
- Notification Service memiliki tabel:
 - notifications (menyimpan pesan notifikasi berdasarkan user_id)
- Auth Service memiliki tabel:
 - users
 - refresh_tokens

Dengan pemisahan ini, setiap service bertanggung jawab atas datanya sendiri dan komunikasi antar service dilakukan melalui API, bukan akses langsung ke database.

HASIL DAN PEMBAHASAN

4.1 Hasil Implementasi Sistem

Berdasarkan hasil pengembangan yang telah dilakukan, sistem pengaduan berbasis microservices berhasil dibangun dengan memisahkan fungsi utama ke dalam beberapa layanan yang saling terintegrasi. Setiap service berjalan pada port yang berbeda dan memiliki tanggung jawab yang spesifik sesuai dengan arsitektur yang dirancang.

Auth Service berhasil mengelola proses autentikasi pengguna, termasuk registrasi, login menggunakan email dan password, serta integrasi OAuth menggunakan Google. Sistem autentikasi menggunakan JSON Web Token (JWT) juga telah berjalan dengan baik, di mana token digunakan sebagai mekanisme validasi akses ke service lain melalui API Gateway.

Complaint Service yang dibangun menggunakan framework Laravel telah mampu menangani proses pembuatan, pengelolaan, serta pengambilan data pengaduan. Fitur pagination dan filtering juga telah diimplementasikan untuk meningkatkan efisiensi dalam pengambilan data.

Disposition Service berhasil menjalankan fungsi sebagai penghubung dalam proses tindak lanjut pengaduan. Service ini dapat memvalidasi keberadaan data pengaduan melalui komunikasi dengan Complaint Service, kemudian menyimpan data disposisi serta melakukan pencatatan aktivitas dalam bentuk log.

Notification Service berfungsi untuk menyimpan notifikasi yang dihasilkan dari aktivitas sistem, khususnya saat terjadi perubahan status atau penerusan pengaduan. Notifikasi ini dibuat secara otomatis melalui komunikasi antar service, sehingga pengguna dapat memperoleh informasi secara real-time.

API Gateway berhasil berperan sebagai pintu masuk utama sistem, yang mengatur routing request ke service yang sesuai serta melakukan validasi token JWT. Dengan adanya gateway, komunikasi antara client dan service menjadi lebih terstruktur dan aman.

4.2 Pembahasan Sistem

Implementasi arsitektur microservices pada sistem ini menunjukkan bahwa pemisahan layanan memberikan fleksibilitas dalam pengembangan dan pengelolaan aplikasi. Setiap service dapat dikembangkan secara independen tanpa mempengaruhi service lain, sehingga mempermudah proses maintenance dan pengembangan fitur di masa depan.

Komunikasi antar service yang dilakukan menggunakan REST API terbukti mampu menghubungkan setiap komponen sistem secara efektif. Sebagai contoh, Disposition Service dapat mengambil data pengaduan dari Complaint Service dan kemudian meneruskan informasi tersebut ke Notification Service untuk menghasilkan notifikasi kepada pengguna.

Selain itu, penggunaan JWT sebagai mekanisme autentikasi memberikan keamanan dalam proses pertukaran data antar service. Token yang dihasilkan oleh Auth Service digunakan untuk memverifikasi setiap request yang masuk melalui API Gateway, sehingga hanya pengguna yang memiliki akses yang dapat menggunakan sistem.

Namun, terdapat beberapa tantangan dalam implementasi sistem ini, seperti pengelolaan komunikasi antar service yang membutuhkan koordinasi yang baik, serta potensi error ketika salah satu service tidak berjalan. Hal ini menunjukkan bahwa meskipun microservices memiliki banyak keunggulan, kompleksitas sistem juga meningkat dibandingkan dengan arsitektur monolitik.

KESIMPULAN

5.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa sistem pengaduan berbasis microservices berhasil dibangun dan berjalan sesuai dengan tujuan yang telah ditetapkan. Sistem ini terdiri dari beberapa layanan utama yang saling terintegrasi, yaitu Auth Service, Complaint Service, Disposition Service, dan Notification Service, yang dihubungkan melalui API Gateway.

Penerapan arsitektur microservices terbukti memberikan keuntungan dalam hal modularitas, fleksibilitas, dan kemudahan pengembangan. Setiap service dapat berjalan secara independen dengan database masing-masing, sehingga mengurangi ketergantungan antar komponen sistem.

Selain itu, implementasi autentikasi menggunakan JWT dan OAuth Google berhasil meningkatkan keamanan dan kemudahan akses bagi pengguna. Komunikasi antar service juga berjalan dengan baik melalui penggunaan REST API, yang memungkinkan pertukaran data secara efisien antar layanan.

Meskipun demikian, sistem microservices memiliki tingkat kompleksitas yang lebih tinggi dibandingkan dengan sistem monolitik, terutama dalam hal pengelolaan komunikasi antar service dan penanganan error. Oleh karena itu, diperlukan perancangan yang baik agar sistem tetap stabil dan mudah dikembangkan.